JANMITRA

Civic Issue Reporting & Management System

Nagpur Municipal Corporation | 4 Categories | 4 Wards | 4 Departments

Team ResolveIT

Software Requirements Specification (SRS)

Detailed technical requirements for developers

CITIZEN PORTAL	Report Issue | Track Status | Verify Resolution
BACKEND (Django)	REST API | Auth | Business Logic | AI Module
AI VALIDATION	Fake Detection (EXIF+ELA+HuggingFace) | YOLO Relevance Check
DATABASE	PostgreSQL + PostGIS | 4 Categories | 4 Wards | 4 Depts
DEPT. DASHBOARD	PWD | Solid Waste | Water Works | Electrical

Anas Shaikh | Anurag Deshmukh | Om Gadekar | Faizan Rayeen

September 2026 | B.Tech Final Year Project

1. System Architecture Overview

JANMITRA follows a three-tier architecture: React frontend, Django REST API backend, and PostgreSQL database. Background AI processing uses Celery and Redis. The system has 4 Django apps: accounts, complaints, dashboard, notifications.

Component	Technology	Role
Frontend	React 18 + Tailwind CSS	Citizen, Worker, Dept. Officer, Admin portals
Backend API	Python Django 4.2 + DRF	All business logic, routing, auth, AI dispatch
Database	PostgreSQL 15 + PostGIS	All data + GPS location + spatial duplicate query
AI -- Fake Detect	Pillow + OpenCV + Transformers	Layer 1: EXIF, ELA, HuggingFace fake detector
AI -- Validation	Ultralytics YOLOv8	Layer 2: civic issue relevance check
Background Tasks	Celery 5 + Redis 7	Async AI processing, notifications, auto-close
Maps	Leaflet + OpenStreetMap	Complaint location pin, ward navigation
Push Notifications	Firebase Cloud Messaging	Browser push for all status changes
Charts	Recharts	Dept. dashboard analytics and trend charts
Deployment	Docker + Render Free Tier	Containerised dev + cloud production hosting

2. Database Schema

2.1 Category Master (Static)

NOTE Only 4 categories exist. These are seeded at deployment and never changed by users.

ID	Name	Department	Auto-routed To
1	Roads & Infrastructure	Public Works Department	PWD queue
2	Garbage & Waste Management	Solid Waste Management Dept.	SWM queue
3	Water & Drainage	Public Health Eng / Water Works	PHE queue
4	Street Lighting & Electrical	Electrical Department	Elec queue

2.2 Ward Master (Static)

4 wards. Seeded at deployment. Citizens and workers are linked to one ward each.

ID	Name	Description
1	Ward 1	NMC zone 1
2	Ward 2	NMC zone 2
3	Ward 3	NMC zone 3
4	Ward 4	NMC zone 4

2.3 users table

Column	Type	Notes

id	UUID PK	Unique identifier
full_name	VARCHAR(100)	User's full name
mobile	VARCHAR(15) UNIQUE	Login identifier (unique)
email	VARCHAR(255)	Optional, for workers/officers
password_hash	TEXT	Django hashed password
role	VARCHAR(20)	citizen / worker / dept_officer / admin
ward_id	FK -> wards	All roles except admin have a ward
department_id	FK -> departments	Workers and dept. officers only
designation	VARCHAR(100)	Workers: e.g. Field Inspector, Supervisor
employee_id	VARCHAR(50)	Workers/officers: NMC employee ID
status	VARCHAR(20)	active / inactive / suspended
fcm_token	TEXT	Firebase push notification token
is_active	BOOLEAN	Account enabled/disabled
registered_at	TIMESTAMP	Account creation date

WARN KEY RULE: Worker's ward_id and department_id BOTH must match the complaint's ward and department for assignment. This is enforced by a database CHECK constraint and API validation.

2.4 departments table

Column	Type	Notes
id	SMALLINT PK	1=PWD, 2=SWM, 3=PHE, 4=Electrical
name	VARCHAR(100)	Full department name
short_name	VARCHAR(20)	PWD / SWM / PHE / ELEC
head_user_id	FK -> users	The dept. officer account for this dept.

2.5 complaints table (Key Fields)

Column	Type	Notes
id	UUID PK	Unique complaint ID
citizen_id	FK -> users	Who reported it
category_id	FK -> categories	1 of 4 categories only
department_id	FK -> departments	Auto-set based on category
ward_id	FK -> wards	Ward where issue is located (1-4)
description	TEXT	Citizen description, min 20 chars
location	Geography(Point)	GPS coordinates (PostGIS)
status	VARCHAR(50)	Complaint lifecycle status
priority_score	FLOAT	Weighted urgency 0-100
priority_level	VARCHAR(10)	Low/Medium/High/Critical
fake_detected	BOOLEAN	True if photo failed fake check

fake_reason	TEXT	Which check failed and why
ai_confidence	FLOAT	YOLO layer 2 confidence (0-1)
is_duplicate	BOOLEAN	Duplicate of another complaint?
parent_complaint_id	FK -> complaints	Original complaint if duplicate
assigned_worker_id	FK -> users	Must match dept+ward of complaint
ward_override	BOOLEAN	True if admin overrode ward rule
override_reason	TEXT	Admin's justification for override
citizen_rating	SMALLINT 1-5	Rating given after closure
submitted_at	TIMESTAMP	Submission date/time
resolved_at	TIMESTAMP	Closure date/time

2.6 ER Diagram (Text Representation)

```
WARD (1)  <---  USERS (many)            -- citizen, worker, officer each in 1 ward
WARD (1)  <---  COMPLAINTS (many)        -- complaint belongs to 1 ward
DEPARTMENT (1)  <---  USERS (many)       -- worker/officer in 1 dept
DEPARTMENT (1)  <---  COMPLAINTS (many)  -- auto-set from category
CATEGORY (1)  <---  COMPLAINTS (many)    -- 1 of 4 categories
CATEGORY (1) --maps--> DEPARTMENT (1)    -- category determines department
USERS/worker (1) <--- COMPLAINTS (many)  -- assigned_worker_id

KEY CONSTRAINT:
  complaints.assigned_worker_id.ward_id MUST EQUAL complaints.ward_id
  complaints.assigned_worker_id.dept_id MUST EQUAL complaints.department_id
  (UNLESS complaints.ward_override = TRUE)

COMPLAINT_IMAGES: complaint_id (FK), image_file, image_type (complaint/proof)
STATUS_HISTORY:   complaint_id (FK), old_status, new_status, changed_by, note
NOTIFICATIONS:    recipient_id (FK), title, body, complaint_id, is_read
```

2.7 Complaint Status Flow

Status	Meaning
Submitted	Citizen just submitted. AI processing queued.
Rejected -- Fake Image	Photo failed Layer 1 fake detection. Citizen notified to resubmit.
Pending Validation	YOLO confidence < 0.5. Dept. officer must review manually.
Validated	Both AI layers passed. Ready for routing.
Routed	Auto-assigned to correct department. Worker assignment pending.
Assigned	Worker assigned (dept+ward match confirmed).
In-Progress	Worker started work on site.
Work Done	Worker uploaded proof. Awaiting citizen verification.
Closed	Citizen accepted OR 72-hour auto-close triggered.
Reopened	Citizen rejected resolution. Back to In-Progress.
Invalid	Dept. officer or admin rejected as false/wrong complaint.

3. Use Cases

UC-01: Citizen Submits Complaint

Actor	Logged-in Citizen
Steps	1. Select 1 of 4 categories 2. Select Ward (1-4) 3. Write description (min 20 chars) 4. Upload 1-3 photos -- each checked for EXIF on upload 5. Confirm GPS location on map 6. Submit
Backend	Complaint saved. Department auto-set from category. Celery task dispatched for AI validation. Complaint ID returned immediately.
Output	Complaint ID shown. Status = Submitted.

UC-02: AI Validation (Background Celery Task)

Trigger	Fires automatically after complaint submission. Citizen does not wait.
Step 1	EXIF check on each photo. No camera metadata? -> Rejected (Fake Image). 0.1 sec.
Step 2	ELA analysis. Too uniform? -> Rejected (Fake Image). 0.5 sec.
Step 3	HuggingFace model. label=artificial with confidence > 0.7? -> Rejected. 3-5 sec.
Step 4	YOLOv8 check. Civic scene detected with confidence >= 0.5? -> Validated. Else -> Pending Validation.
Step 5	Duplicate check: same category + same ward + GPS within 100m using PostGIS.
Step 6	Priority score calculated. Complaint routed to department.
Output	Status updated. Citizen notified via Firebase push.

UC-03: Dept. Officer Assigns Worker

Actor	Dept. Officer (sees only their dept.)
Steps	1. View unassigned complaints in their dept. 2. Select a complaint 3. System shows available workers: same dept. + same ward as complaint 4. Officer selects a worker and clicks Assign 5. Worker receives push notification
Validation	API enforces: worker.department == complaint.department AND worker.ward == complaint.ward. Returns 400 if mismatch (unless admin override).
Override	Admin can assign any worker to any complaint. System records override flag and requires a reason.

UC-04: Worker Handles Task

Actor	Worker (sees only dept + ward complaints)
Task List	Sorted by priority (Critical first). Shows category, ward, description, photos, GPS map.
Flow	Accept -> In-Progress -> Upload proof photo(s) -> Mark Work Done
Rule	Mark Work Done is disabled until at least 1 proof photo uploaded. Enforced frontend + backend.
Output	Status = Work Done. Citizen notified.

UC-05: Dept. Officer Views Dashboard

Actor	Dept. Officer (filtered to their department only)
KPIs	Total complaints, Pending, In-Progress, Resolved, Rejected, Avg resolution time
Charts	Severity distribution pie chart, Ward-wise complaint bar chart, Trend line chart
Tables	Worker workload table, High-priority open complaints list
Filters	Date range, Ward (1-4), Status, Severity/Priority level
Admin	Admin sees same dashboard but can switch between all 4 departments + cross-dept view

4. API Endpoint List

Method	Endpoint	Role	Purpose
POST	/api/v1/auth/register/	Public	Register citizen
POST	/api/v1/auth/login/	All	Login, get JWT
POST	/api/v1/auth/refresh/	All	Refresh access token
GET	/api/v1/categories/	All	List 4 categories
GET	/api/v1/wards/	All	List 4 wards
POST	/api/v1/complaints/	Citizen	Submit complaint

GET	/api/v1/complaints/	Citizen/Admin	List complaints
GET	/api/v1/complaints/{id}/	All	Complaint detail
PATCH	/api/v1/complaints/{id}/	Worker/Officer	Update status
POST	/api/v1/complaints/{id}/assign/	Officer/Admin	Assign worker
POST	/api/v1/complaints/{id}/images/	Worker	Upload proof photo
POST	/api/v1/complaints/{id}/verify/	Citizen	Accept/reject resolution
GET	/api/v1/worker/tasks/	Worker	My task list (dept+ward)
GET	/api/v1/worker/available/?complaint_id=	Officer/Admin	Available workers for this complaint
GET	/api/v1/dashboard/kpi/	Officer/Admin	KPI summary
GET	/api/v1/dashboard/ward-stats/	Officer/Admin	Ward-wise complaint counts
GET	/api/v1/dashboard/trends/	Officer/Admin	Complaint trend over time
GET	/api/v1/dashboard/workers/	Officer/Admin	Worker workload table
GET	/api/v1/dashboard/high-priority/	Officer/Admin	Critical+High open complaints
GET	/api/v1/dashboard/pending-validation/	Officer/Admin	Low-confidence AI queue
PATCH	/api/v1/dashboard/complaints/{id}/validate/	Officer/Admin	Manual validate/reject
GET	/api/v1/admin/workers/	Admin	All workers across depts+wards
POST	/api/v1/admin/workers/	Admin	Register new worker
PATCH	/api/v1/admin/workers/{id}/	Admin	Update worker details